

HabitHive – Application Documentation

Part 2: App Prototype Development
Module: Open Source Coding (Intermediate) – OPSC6312
Katleho Khutsoane | Student Number: ST10442069

GitHub repository:	https://github.com/KAT23A1/HabitHive
Demonstration video:	https://youtu.be/GbiZSelt0oE

1. Introduction and Purpose

HabitHive is a mobile habit-tracking application for Android, developed for Part 2 of the OPSC6312 Portfolio of Evidence. Its purpose is to help users build and maintain daily habits by making logging effortless and progress visible. A user registers a secure account, creates habits, marks them complete each day to build streaks, and reviews their consistency through a statistics screen. The application was built in Kotlin (JetBrains, 2026) using Jetpack Compose (Android Developers, 2026a) for the user interface, and it connects to a hosted cloud database through a REST API. The design of the application was defined in the Part 1 planning document, and this document explains what was built and how.

2. Features Implemented

- **Secure sign-in:** users register and log in with an email and password. Passwords are hashed by the authentication service and never stored in plain text.
- **Habit management (CRUD):** users can create, edit and delete habits, each stored in the cloud database.
- **Daily logging and streaks:** a habit can be ticked complete for the day, and the app calculates a running streak and a habit score that rewards consistency.
- **Statistics:** the app displays the habit score, best streak, and a weekly completion chart built from the logged data.
- **Settings:** a dark/light theme toggle that changes the whole app and is remembered between sessions, account information, and logout.
- **Robust interface:** input is validated, deletes are confirmed, and a lost connection is handled gracefully with a retry option rather than crashing.

3. Technology Stack

Layer	Technology
Language	Kotlin (JetBrains, 2026)
User interface	Jetpack Compose, Material 3 (Android Developers, 2026a)
Architecture	MVVM – ViewModel and Repository (Android Developers, 2026c)
Networking	Retrofit and OkHttp (Square, 2026a; Square, 2026b)

Backend	Supabase – PostgreSQL, REST API and Auth (Supabase, 2026a)
Testing / CI	JUnit (JUnit, 2026) and GitHub Actions (GitHub, 2026)

4. Design Considerations

Choice of backend. Supabase (Supabase, 2026a) was chosen because it provides a hosted PostgreSQL database, an automatically generated REST API, and an authentication service in one platform. This meant the application could connect to a real, online REST backend without hosting a separate server.

REST integration. The application uses Retrofit (Square, 2026a) to call Supabase's REST endpoints using the standard HTTP verbs GET, POST, PATCH and DELETE, with JSON request and response bodies (Supabase, 2026b). Authentication is handled through Supabase's Auth endpoints, which return a token that is attached to every subsequent request.

Security. Row Level Security is enabled on every table so that each user can only read and write their own data. API keys are kept out of version control using a local properties file and exposed to the app through the generated BuildConfig class.

Architecture and testability. The application follows the recommended MVVM pattern (Android Developers, 2026c): the Compose user interface observes a ViewModel, which uses a repository and network layer for data. Asynchronous work uses Kotlin coroutines. The streak calculation is written as a pure function so that it can be unit-tested independently of the Android framework.

5. How the REST API Works

When a user signs in, the app authenticates against Supabase Auth and receives a token. That token, together with the project API key, is attached to every data request. Data operations then use the REST API: for example, POST /rest/v1/habits creates a habit, GET /rest/v1/habit_logs retrieves completions, and DELETE removes a record. All requests and responses are JSON (Supabase, 2026b).

6. Testing and GitHub Actions

Unit tests for the streak logic were written using JUnit (JUnit, 2026) and are located in the project's test source set. A GitHub Actions workflow (GitHub, 2026) runs automatically on every push: it installs JDK 17, runs the unit tests, and builds the application to confirm it compiles. Successful runs are visible under the repository's Actions tab, and the project was committed and pushed to GitHub throughout development.

7. Conclusion

HabitHive delivers a working Android prototype that meets the required features: secure authentication, a settings menu, a hosted REST API and database, and three user-defined features (habit management, daily logging with streaks, and statistics). The source code, README, and automated tests are available in the linked repository, and the linked video demonstrates the application running on a physical device.

8. References

- Android Developers. 2026a. *Jetpack Compose*. [Online]. Available at: <https://developer.android.com/develop/ui/compose> [Accessed: 28 September 2026].
- Android Developers. 2026b. *Save data in a local database using Room*. [Online]. Available at: <https://developer.android.com/training/data-storage/room> [Accessed: 28 September 2026].
- Android Developers. 2026c. *Guide to app architecture*. [Online]. Available at: <https://developer.android.com/topic/architecture> [Accessed: 28 September 2026].
- GitHub. 2026. *GitHub Actions documentation*. [Online]. Available at: <https://docs.github.com/en/actions> [Accessed: 28 September 2026].
- JetBrains. 2026. *Kotlin documentation*. [Online]. Available at: <https://kotlinlang.org/docs/home.html> [Accessed: 28 September 2026].
- JUnit. 2026. *JUnit 4*. [Online]. Available at: <https://junit.org/junit4/> [Accessed: 28 September 2026].
- Square. 2026a. *Retrofit*. [Online]. Available at: <https://square.github.io/retrofit/> [Accessed: 28 September 2026].
- Square. 2026b. *OkHttp*. [Online]. Available at: <https://square.github.io/okhttp/> [Accessed: 28 September 2026].
- Supabase. 2026a. *Supabase documentation*. [Online]. Available at: <https://supabase.com/docs> [Accessed: 28 September 2026].
- Supabase. 2026b. *REST API*. [Online]. Available at: <https://supabase.com/docs/guides/api> [Accessed: 28 September 2026].